

CS151 Intro to Data Structures

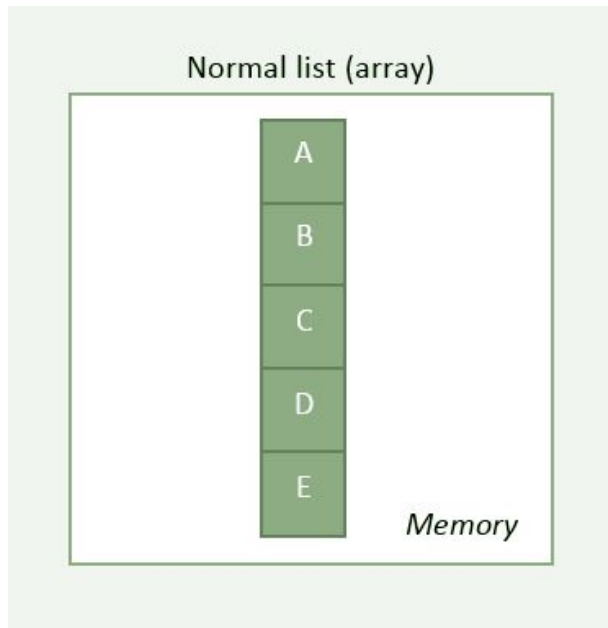
LinkedLists II

Announcements

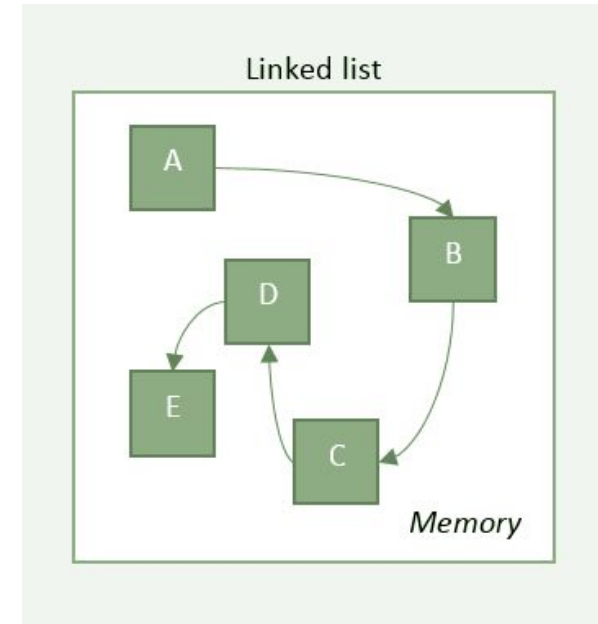
- Quiz Today in Lab
 - Exceptions and Inheritance
- HW1 due Friday
- HW2 RELEASED
 - due Friday Oct 2nd
 - prior students reported an average of 20 hours!

List versus Array - memory

An array is a single consecutive piece of memory



A list can be made of many disjoint pieces



Access Operation

Let's review the code we wrote

- Runtime analysis
 - $O(n)$

Insert Operation

Let's review the code we wrote

- Runtime analysis
 - Insert at head?
 - $O(1)$
 - Insert at tail?
 - $O(n)$
 - Insert at arbitrary location? (middle of list)
 - $O(n)$

Insert Operation

What if we keep a pointer to the tail?

```
private Node tail;
```

How does this change our insertTail method?

- Runtime analysis
 $O(1)$

Exercise: swapEnds

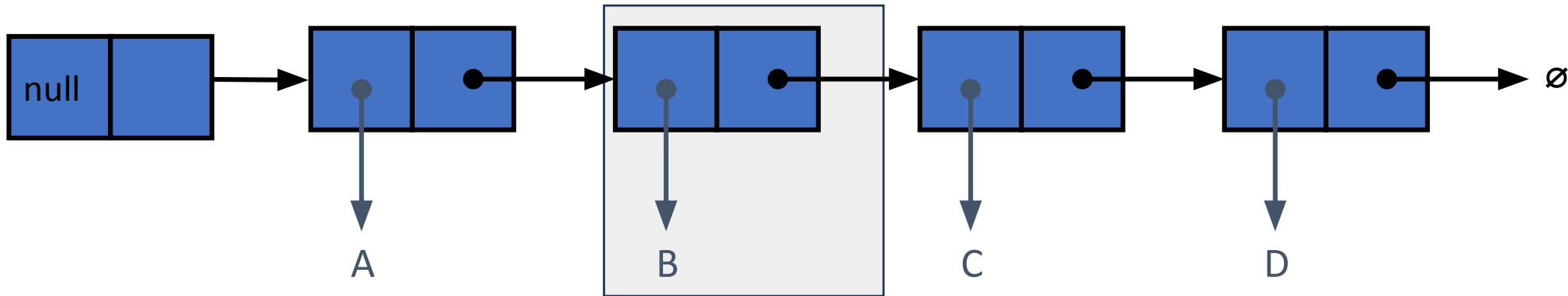
Write a LinkedList class method that swaps the first two nodes and the last two nodes. You may assume the list has at least 4 nodes. The method must be **in place** (do not allocate any additional nodes).

Remove Operation

- Let's write it on the board quickly

Remove Operation `remove("B")`

head



head



Properties in LinkedList

What do we need to keep track of?

- Head
- Number of elements (optional)

Quiz

Rank from most efficient to least efficient:

- LinkedList find
- ExpandableArray find
- LinkedList insert at beginning
- ExpandableArray insert at beginning

Exercise: Comparing Linked Lists

Write a method that determines whether two linked lists contain the same strings in the same order.

```
public static boolean areEqual(Node list1, Node list2) {  
    ....  
}
```

Comparing Strings in Java

`compareTo()` method is used to compare two strings lexicographically.

```
int result = str1.compareTo(str2);
```

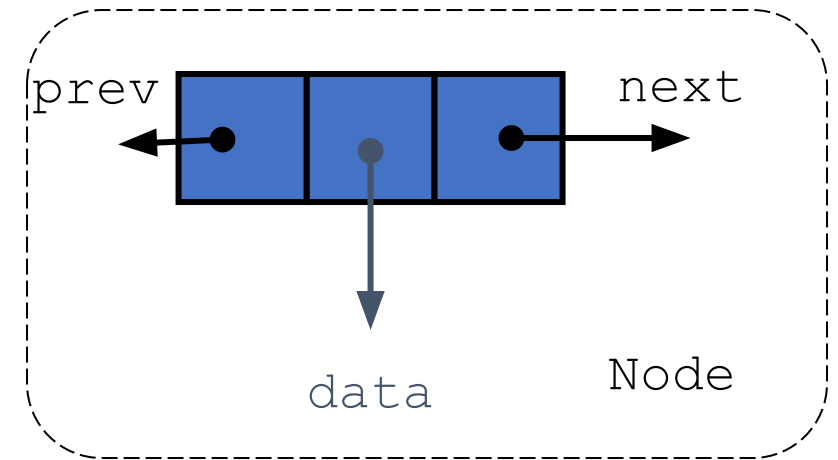
It compares two strings character by character based on their Unicode values.
The method returns:

- `0` if `str1` is equal to `str2`
- A **negative value** if `str1` is lexicographically smaller than `str2`
- A **positive value** if `str1` is lexicographically greater than `str2`

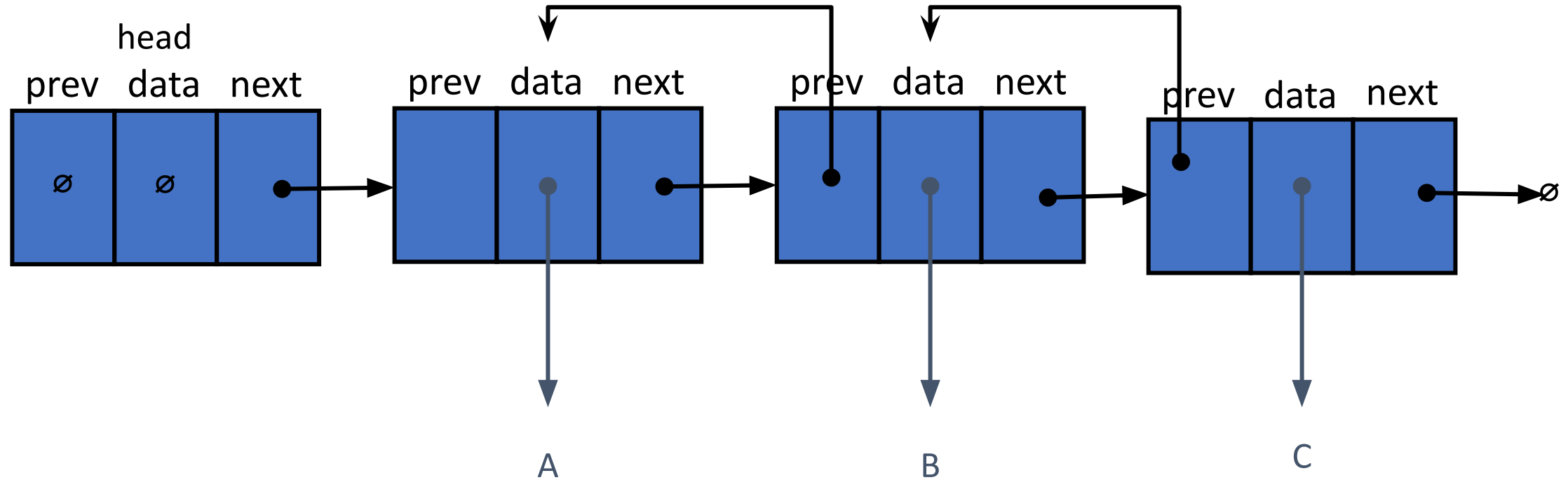
Doubly Linked Lists

A node

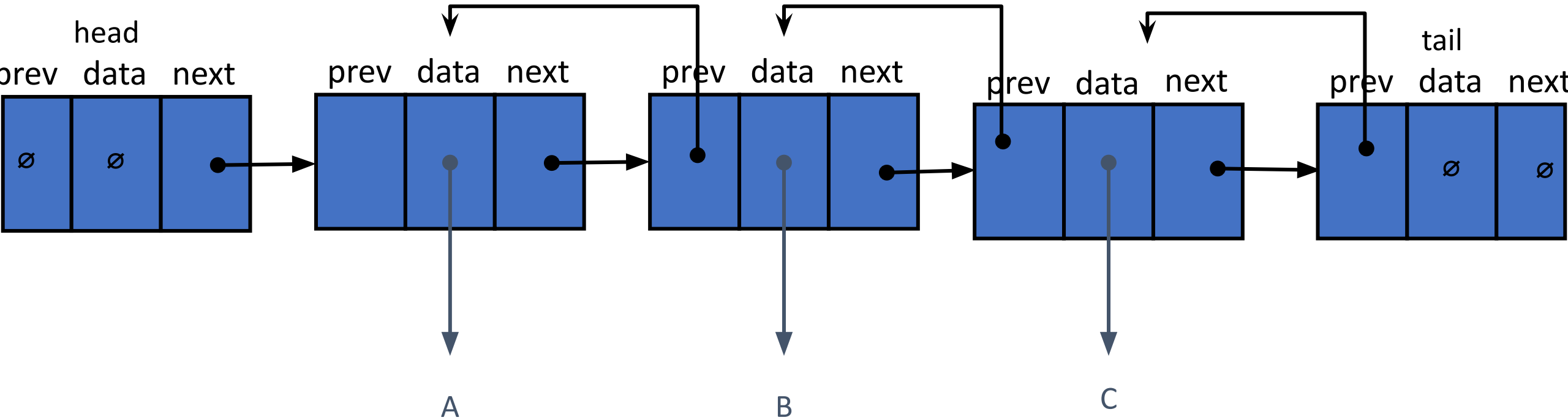
```
public class Node<T> {  
    private T data;  
    private Node next;  
    private Node prev;  
}
```



Doubly Linked List



Doubly Linked List



Exercise

Write a `moveToFront(String data)` method. It should move the node containing the specified string data to the front of the doubly linked list. The node is assumed to be present in the list. This method updates the head of the list to point to the moved node and adjusts all necessary links in the list to maintain the structure.

Draw a “test” Doubly Linked List and execute your method on it.

```
public class DoublyLinkedList {
    private Node head;
    private Node tail;
    private int size;

    public DoublyLinkedList() {
        head = new Node();
        tail = new Node();

        head.next = tail;
        tail.prev = head;
    }

    public class Node {
        public String data;
        private Node next;
        private Node prev;

        public Node(String data) {
            this.data = data;
        }
    }

    public void moveToFront(String data) {
    }
}
```

Exercise 2

Assume correct `insert` and `expand` methods exist. Write the `moveToFront` method.

It should move the specified element to the front of the expandable array. It should search for the first occurrence of the given element in the array. Upon finding it, the method shifts all elements between the start of the array and the found element one position to the right. The order of the remaining elements should be preserved.

Draw 3 “test” arrays and execute your method on them. Again, make sure to consider edge cases.

```
public class ExpandableArray<E> {  
    private E[] data;  
    private int size;  
  
    public void moveToFront(E data) {  
  
    }  
  
}
```

Algorithmic Analysis

What is the big-o notation of the following operations?

1. `moveToFront` - `DoublyLinkedList`
2. `moveToFront` - `ExpandableArray`

Compare the two operations. Is one cheaper? Why or why not?

Summary

- DoublyLinkedList:
 - tail member variables
 - Nodes have a prev member variable
- Allows for reverse traversal
- Allows for more efficient insert at back (tail)
 - $O(n)$ on SLL
 - $O(1)$ on DLL